

Assignment 3 | Bridge Pattern - Short Report

Student	Zhakanov Sanzhar
Group	SE-2526
Topic	A - Drawing
Base commit	c9c84c6
Repository	https://github.com/lonelysun21/SDP_3_assignment

1. Purpose and why Bridge fits

The application models two independent dimensions. The first dimension is the abstraction hierarchy: Shape with the refined abstractions Circle and Square. The second dimension is the implementation hierarchy: Renderer with VectorRenderer, RasterRenderer, and AsciiRenderer. Bridge is appropriate because both dimensions can change independently and are connected through composition instead of creating a subclass for every shape/renderer combination.

The bridge is the field **renderer: Renderer** stored in Shape. Shape receives that interface reference through its constructor. Circle and Square call low-level rendering operations through the interface, so they do not depend on any concrete renderer class.

2. UML class diagram

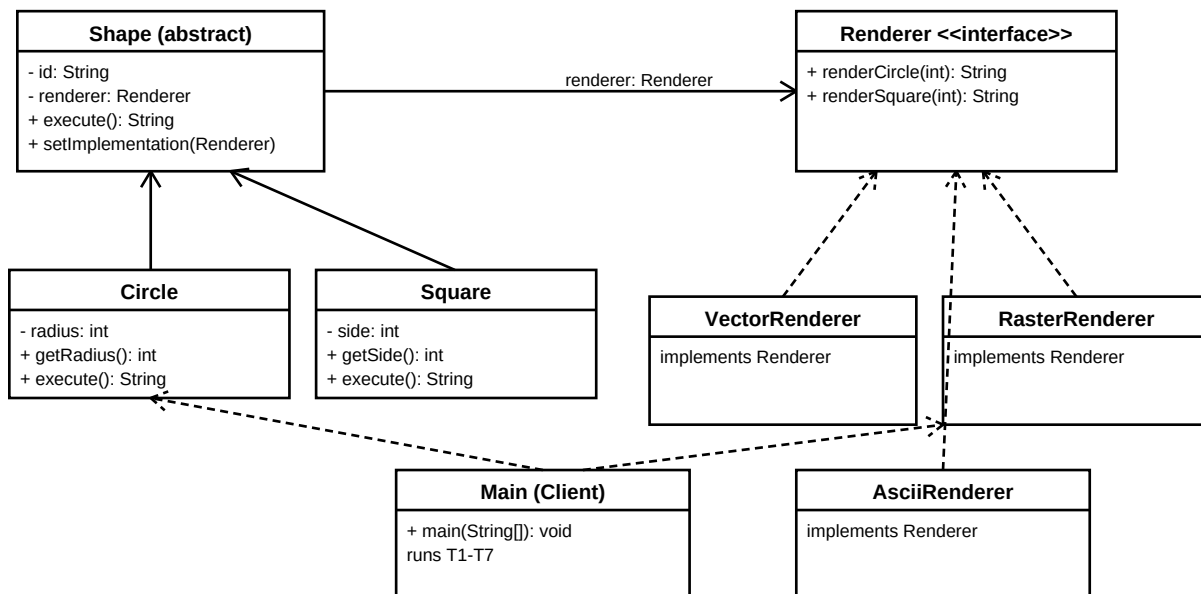


Figure 1. Final Bridge structure used by the project. The interface-typed association from Shape to Renderer is the bridge between the two hierarchies.

3. Implementation and Clean Code examples

Separation of responsibilities

```
public abstract class Shape {  
    private String id;  
    protected Renderer renderer;  
    public abstract String execute();  
}
```

Benefit: Shape keeps abstraction-side state and the bridge reference. Rendering details stay in Renderer implementations, so high-level shape behavior and low-level rendering behavior are separated.

Meaningful names

```
public interface Renderer {  
    String renderCircle(int radius);  
    String renderSquare(int side);  
}
```

Benefit: Names such as Renderer, renderCircle, radius, and renderSquare directly describe the roles and data used by the program.

Small focused methods

```
public String execute() {  
    return renderer.renderCircle(radius);  
}
```

Benefit: Circle.execute() has one focused responsibility: delegate Circle rendering through the bridge. It does not contain concrete rendering logic.

Avoidance of duplication

```
circle1.setImplementation(new RasterRenderer());  
String after = circle1.execute();
```

Benefit: Runtime replacement reuses the same Circle and the same execute() workflow. There is no separate VectorCircle or RasterCircle subclass.

Encapsulation of state

```
private String id;  
private int radius;  
  
public String getId() { return id; }  
public int getRadius() { return radius; }
```

Benefit: Domain state is private and is exposed only through controlled accessors. T5 can verify that the state remains unchanged without exposing fields directly.

4. Runtime switching (T5)

T5 creates one Circle with VectorRenderer, stores a second reference to that same object, records its ID and radius, then calls setImplementation(new RasterRenderer()). The program checks reference equality with == and verifies that the ID and radius are unchanged. The object is therefore the same abstraction object, while only the implementation reference changes. The result changes from vector rendering to raster rendering.

5. Independent extension with I3

The initial working version contained VectorRenderer (I1), RasterRenderer (I2), T1-T4, and the runtime switch T5. That version was committed as base commit **c9c84c6**. Afterwards, AsciiRenderer (I3) was added as a new implementation of Renderer and Main was updated to demonstrate T6 and T7.

The existing abstraction classes (Shape, Circle, Square), the Renderer interface, VectorRenderer, and RasterRenderer remain unchanged. This demonstrates independent extensibility: a new low-level implementation can be added without changing the abstraction hierarchy.

The required source diff is generated with:

```
git diff c9c84c6 HEAD -- src > extension.diff
```

6. Trade-off

A specific trade-off of Bridge is increased structural complexity. The design introduces an abstraction hierarchy, an implementor interface, and multiple implementation classes. For a very small program with only one implementation, this extra indirection could be unnecessary. In this assignment, however, the separate hierarchies and runtime implementation switching justify the structure.

7. Bridge vs Adapter

Bridge and Adapter can both use composition, but their intent is different. Bridge is designed in advance to separate an abstraction from its implementation so both can vary independently. Adapter is normally used when existing classes have incompatible interfaces and a translating wrapper is required so they can work together. In this project, no incompatible interface is being repaired; Shape and Renderer are deliberately designed as two cooperating hierarchies, so Bridge is the correct intent.

8. Demonstration checks

Check	Action	Required evidence
T1	Circle + VectorRenderer	radius 2
T2	Circle + RasterRenderer	radius 2
T3	Square + VectorRenderer	side 3
T4	Square + RasterRenderer	side 3
T5	same Circle; Vector -> Raster	same object, state unchanged
T6	Circle + AsciiRenderer	radius 2
T7	Square + AsciiRenderer	side 3

9. References

1. Almas Ospanov, Lecture 4: The Structural Design Pattern - Bridge Pattern, Software Design Patterns course.
2. Assignment 3 | Bridge Pattern Instructions, Astana IT University, 2026-2027.
3. Refactoring.Guru, Bridge design pattern (conceptual reference listed by the course materials).